Spring Security

Complete Developer Documentation

JWT Authentication & Authorization

Spring Boot 4 | Spring Flyway Migration | Lombok | MySQL | JDBC Template

Source Code: https://github.com/PasinduOG/spring-security

Pasindu Owa Gamage

1. Overview & Architecture

This documentation covers a complete Spring Security implementation using JWT (JSON Web Tokens) for stateless authentication. The application uses Spring Boot 4, Flyway for database migrations, and Lombok for boilerplate reduction.

1.1 Technology Stack

Technology	Purpose & Version
Spring Boot 4	Core framework — auto-configuration, embedded server, dependency injection
Spring Security 6+	Authentication and authorization framework — filter chain, security context
JJWT (io.jsonwebtoken)	JWT token generation, signing, parsing, and validation
Spring Flyway	Database schema version control — manages SQL migrations automatically
Lombok	Reduces boilerplate — @Getter, @Setter, @RequiredArgsConstructor
JdbcTemplate	Raw SQL access via Spring — used in UserRepositoryImpl instead of JPA
BCrypt	Password hashing algorithm — strength factor 12 used here

1.2 Request Flow Diagram

Every HTTP request travels through this pipeline before reaching your controller:

```
HTTP Request
    |
    ▼
JwtAuthFilter  (OncePerRequestFilter)
    ├── No 'Authorization: Bearer ...' header? → pass through (public routes)
    ├── Extract JWT from header
    ├── extractUsername(jwt) → username from Claims subject
    ├── No SecurityContext auth yet? → loadUserByUsername(username)
    ├── isTokenValid(jwt, userDetails)? → set Authentication in SecurityContext
    |
    ▼
SecurityFilterChain rules
    ├── /api/auth/**     → permitAll  (no auth needed)
    ├── /swagger-ui/**   → permitAll
    ├── /admin/**        → ROLE_ADMIN only
    ├── /user/**         → ROLE_USER only
    └── everything else  → authenticated()
    |
    ▼
DaoAuthenticationProvider
    ├── CustomUserDetailsService.loadUserByUsername()
    └── BCryptPasswordEncoder.matches()
    |
    ▼
Your @RestController / @Service
```

2. Project Structure

The project follows a layered architecture separating concerns cleanly:

```
src/main/java/io/og4dev/
├── config/
│   └── SecurityConfig.java          ← Security filter chain, beans
├── controller/
│   └── AuthController.java          ← /api/auth/register, /api/auth/login
├── dto/
│   ├── AuthResponse.java            ← Token + user info returned to client
│   ├── LoginRequestDto.java         ← Login request payload
│   └── RegisterRequestDto.java      ← Registration request payload
├── entity/
│   └── UserEntity.java              ← Domain model (no JPA annotations)
├── filter/
│   └── JwtAuthFilter.java           ← JWT extraction + SecurityContext setup
├── repository/
│   ├── UserRepository.java          ← Interface
│   └── impl/
│       └── UserRepositoryImpl.java  ← JdbcTemplate SQL implementation
├── service/
│   ├── AuthService.java             ← Interface
│   ├── CustomUserDetailsService.java← Loads UserDetails for Spring Security
│   ├── JwtService.java              ← Interface
│   └── impl/
│       ├── AuthServiceImpl.java     ← Register + Login logic
│       └── JwtServiceImpl.java      ← Token generation + validation
└── util/
    └── Role.java                    ← Enum: ROLE_ADMIN, ROLE_USER

src/main/resources/
├── application.yml                  ← DB + JWT config
└── db/migration/
    └── V1__create_users_table.sql   ← Flyway migration
```

3. Security Configuration — SecurityConfig.java

The SecurityConfig class is the heart of the security setup. It defines what URLs are accessible, configures the authentication mechanism, and wires the JWT filter into the chain.

3.1 Class Annotations

Annotation	What it does
@Configuration	Marks this as a Spring configuration class — Spring scans it for @Bean methods
@EnableWebSecurity	Activates Spring Security's web security support and the filter chain
@EnableMethodSecurity	Enables @PreAuthorize, @PostAuthorize, @Secured on individual methods
@RequiredArgsConstructor	Lombok — generates a constructor injecting all final fields (DI)

3.2 Full Code with Annotations

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;
    private final CustomUserDetailsService service;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) {
        http
          .csrf(AbstractHttpConfigurer::disable)    // Disable CSRF — stateless JWT
doesn't need it
          .sessionManagement(session ->
              session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))    // No
server-side sessions
          .authorizeHttpRequests(auth -> auth
              .requestMatchers("/api/auth/**").permitAll()    // Login & register are
public
              .requestMatchers("/v3/api-docs/**").permitAll()    // OpenAPI spec public
              .requestMatchers("/swagger-ui/**").permitAll()    // Swagger UI public
              .requestMatchers("/admin/**").hasAuthority("ROLE_ADMIN")    // Admin only
              .requestMatchers("/user/**").hasAuthority("ROLE_USER")    // User only
              .anyRequest().authenticated()    // All others require valid JWT
          )
          .authenticationProvider(authenticationProvider())    // Use DAO provider
          .addFilterBefore(jwtAuthFilter,
              UsernamePasswordAuthenticationFilter.class);    // JWT runs before default
auth filter
        return http.build();
    }
}
```

3.3 Why CSRF is Disabled

Key Concept: CSRF (Cross-Site Request Forgery) protection is designed for cookie-based session authentication. Since this app uses stateless JWT in the Authorization header, there are no session cookies to forge. Disabling CSRF is safe and required for REST APIs.

3.4 Authentication Beans

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider(service);   //
Pass UserDetailsService
    provider.setPasswordEncoder(passwordEncoder());   // Use BCrypt for password matching
    return provider;
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) {
    return config.getAuthenticationManager();   // Used by AuthService to authenticate
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);   // 12 = work factor (higher = slower = safer)
}
```

3.5 BCrypt Strength Factor

Strength	Approx. hashing time
10 (default)	~100ms — fine for low-traffic apps
12 (used here)	~400ms — good balance of security/speed
14	~1.5s — for high-security apps

4. JWT Authentication Filter — JwtAuthFilter.java

This filter intercepts every HTTP request exactly once (OncePerRequestFilter) and authenticates the user if a valid JWT is present.

4.1 Full Code with Inline Explanation

```
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService service;
    private final CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");   // Read the
Authorization header

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {   // No JWT? Pass
to next filter
            filterChain.doFilter(request, response);
            return;
        }

        String jwt = authHeader.substring(7);   // Strip 'Bearer ' prefix (7 chars)
        String username;
        try {
            username = service.extractUsername(jwt);   // Decode the 'sub' claim
        } catch (Exception e) {
            filterChain.doFilter(request, response);   // Invalid/expired token → ignore
            return;
        }

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {   // Not
already authenticated?
            UserDetails userDetails =
                userDetailsService.loadUserByUsername(username);   // Load from DB by
username
            if (service.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());   // null = no
credentials needed
                authToken.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(request));   //
Attach IP, session info
                SecurityContextHolder.getContext()
                    .setAuthentication(authToken);   // Mark user as authenticated
            }
        }

        filterChain.doFilter(request, response);   // Always continue the chain
    }
}
```

4.2 Why Check SecurityContextHolder Before Loading User?

Performance Note: Checking SecurityContextHolder.getContext().getAuthentication() == null prevents re-authenticating an already authenticated request. This avoids an unnecessary database hit on every filter pass within the same request lifecycle.

4.3 Flow Summary

- Request arrives → read Authorization header
- If no 'Bearer ' prefix → skip filter (public route or missing token)
- Extract JWT → parse username from 'sub' claim
- Load UserDetails from database using username
- Validate token (username match + not expired)
- If valid → set UsernamePasswordAuthenticationToken in SecurityContextHolder
- Continue filter chain regardless

5. JWT Service — JwtServiceImpl.java

The JwtService handles all JWT operations: generating tokens, extracting claims, and validating tokens. It uses the JJWT library (io.jsonwebtoken).

5.1 Configuration

```
# application.yml
app:
  jwt:
    secret: ${JWT_SECRET}    // Base64-encoded 256-bit secret (env var)
    expiration: ${JWT_EXPIRE}    // Milliseconds, e.g. 86400000 = 24h
```

Security: Never hardcode the JWT secret. Always use environment variables. Generate a strong secret with: openssl rand -base64 64

5.2 Token Generation

```
@Value("${app.jwt.secret}")
private String secretKey;

@Value("${app.jwt.expiration}")
private long expiration;

public String generateToken(UserDetails userDetails) {
    Map<String, Object> extraClaims = new HashMap<>();
    extraClaims.put("role", getFirstAuthority(userDetails.getAuthorities()));   // Add
role claim
    return Jwts.builder()
        .claims(extraClaims)   // Custom claims (role)
        .subject(userDetails.getUsername())   // 'sub' — standard JWT subject
        .issuedAt(new Date(System.currentTimeMillis()))   // 'iat' — issued at
        .expiration(new Date(System.currentTimeMillis() + expiration))   // 'exp' —
expiry
        .signWith(getSigningKey())   // HMAC-SHA256 signing
        .compact();
}
```

5.3 Token Validation

```
public boolean isTokenValid(String token, UserDetails userDetails) {
    return extractUsername(token).equals(userDetails.getUsername())
        && !isTokenExpired(token);   // Both conditions must be true
}

public boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());   // expiry date < now?
}
```

5.4 Claims Extraction

```
public Claims extractAllClaims(String token) {
    return Jwts.parser()
        .verifyWith(getSigningKey())   // Verify signature first
        .build()
        .parseSignedClaims(token)
        .getPayload();   // Returns all claims as Claims object
}

public String extractUsername(String token) {   // Gets the 'sub' claim
    return extractAllClaims(token).getSubject();
}

public String extractRole(String token) {   // Gets our custom 'role' claim
    return (String) extractAllClaims(token).get("role");
}

public SecretKey getSigningKey() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretKey));   // Decode Base64
secret → HMAC key
}
```

5.5 JWT Structure Reference

Claim	Description
sub	Subject — the username. Used to identify the user.
iat	Issued At — timestamp when token was created (milliseconds since epoch).
exp	Expiration — timestamp when token becomes invalid. Filter checks this.
role	Custom claim — the user's role (e.g. ROLE_ADMIN, ROLE_USER). Added manually.

6. Authentication Service — AuthServiceImpl.java

AuthServiceImpl handles two operations: registering new users and logging in existing ones. It depends on JwtService, PasswordEncoder, and AuthenticationManager.

6.1 Register Flow

```
public AuthResponse register(RegisterRequestDto request) {
    if (userRepository.existsByUsername(request.getUsername()))   // Prevent duplicate
usernames
        throw new IllegalArgumentException("Username already taken");
    if (userRepository.existsByEmail(request.getEmail()))   // Prevent duplicate emails
        throw new IllegalArgumentException("Email already registered");

    Role role = (request.getRole() != null)
        ? request.getRole() : Role.ROLE_USER;   // Default to ROLE_USER if not specified

    UserEntity entity = new UserEntity();
    entity.setUsername(request.getUsername());
    entity.setEmail(request.getEmail());
    entity.setPassword(passwordEncoder.encode(request.getPassword()));   // NEVER store
plain text!
    entity.setRole(role);
    entity.setEnabled(true);   // Account enabled on registration
    userRepository.save(entity);

    UserDetails details = userDetailsService.loadUserByUsername(entity.getUsername());
    String token = jwtService.generateToken(details);   // Return token immediately on
register
    return new AuthResponse(token, entity.getUsername(), role);
}
```

6.2 Login Flow

```
public AuthResponse login(LoginRequestDto request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            request.getUsername(), request.getPassword()    // Spring validates
credentials
        )
    );   // Throws BadCredentialsException if invalid

    UserDetails details =
        userDetailsService.loadUserByUsername(request.getUsername());    // Load fresh
UserDetails
    String token = jwtService.generateToken(details);
    Role role = getRoleFromAuthorities(details.getAuthorities());
    return new AuthResponse(token, request.getUsername(), role);
}
```

Why call authenticationManager.authenticate()? This is the standard Spring Security way to validate credentials. It calls DaoAuthenticationProvider → loadUserByUsername → BCrypt password check. If invalid, it throws BadCredentialsException automatically — no manual password comparison needed.

6.3 AuthResponse — What Gets Returned

```
{
    "token": "eyJhbGciOiJIUzI1NiJ9...",    // JWT to store client-side
    "tokenType": "Bearer",    // Always 'Bearer' for JWT
    "username": "john_doe",    // The authenticated username
    "role": "ROLE_USER"    // The user's role
}
```

7. User Repository — UserRepositoryImpl.java

Instead of JPA/Hibernate, this implementation uses Spring's JdbcTemplate for direct SQL access. This keeps the dependency footprint small and gives full control over queries.

7.1 Key Methods

```
@Repository
@RequiredArgsConstructor
public class UserRepositoryImpl implements UserRepository {
    private final JdbcTemplate template;

    @Override
    public UserEntity findByUsername(String username) {
        try {
            return template.queryForObject(
                "SELECT * FROM users WHERE username=?",   // Parameterized query (SQL
injection safe)
                (rs, rowNum) -> new UserEntity(
                    rs.getLong("id"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("password"),
                    Role.valueOf(rs.getString("role")),   // Convert String → enum
                    rs.getBoolean("enabled")
                ), username);
        } catch (EmptyResultDataAccessException e) {
            return null;   // No user found → return null (not an exception)
        }
    }

    @Override
    public UserEntity save(UserEntity entity) {
        template.update(
            "INSERT INTO users (username, email, password, role, enabled) VALUES
(?,?,?,?,?)",
            entity.getUsername(),
            entity.getEmail(),
            entity.getPassword(),
            entity.getRole().name(),   // Enum → String for storage
            entity.getEnabled());
        return findByUsername(entity.getUsername());   // Re-fetch to get generated ID
    }
}
```

Note: EmptyResultDataAccessException is Spring's way of signaling no rows returned from queryForObject. We catch it and return null instead of propagating the exception — existsByUsername/existsByEmail use null-check on findByUsername.

8. DTOs, Entity & Validation

8.1 UserEntity

A plain Java object (no JPA) representing a row in the users table. Flyway creates the table structure.

```
@Getter @Setter
@AllArgsConstructor @NoArgsConstructor
public class UserEntity {
    private Long id;
    private String username;
    private String email;
    private String password;   // Always BCrypt hashed — never plain text
    private Role role;
    private Boolean enabled;   // false = account locked/disabled
}
```

8.2 RegisterRequestDto — Validation Annotations

```
@Getter @Setter @NoArgsConstructor
public class RegisterRequestDto {
    @NotBlank(message = "Username is required")   // Cannot be null or whitespace
    @Size(min = 3, max = 50)
    private String username;

    @NotBlank(message = "Email is required")   // Separate from @Email
    @Email(message = "Please enter a valid email")   // Validates email format
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 6, max = 100)   // Enforce minimum password length
    private String password;

    private Role role;   // Optional — defaults to ROLE_USER if null
}
```

8.3 LoginRequestDto

```
@Getter @Setter @NoArgsConstructor
public class LoginRequestDto {
    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;
}
```

8.4 AuthResponse

```
@Getter @Setter @NoArgsConstructor
public class AuthResponse {
    private String token;
    private String tokenType;   // Always 'Bearer'
    private String username;
    private Role role;

    public AuthResponse(String token, String username, Role role) {
        this.token = token;
        this.tokenType = "Bearer";   // Set automatically — client uses this prefix
        this.username = username;
        this.role = role;
    }
}
```

9. Flyway Database Migration

Flyway manages database schema evolution automatically. On startup, it scans resources/db/migration for SQL scripts and runs any that haven't been applied yet.

9.1 application.yml Configuration

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/spring_security_db?createDatabaseIfNotExist=true
// Auto-create DB
    username: root
    password: ${DB_PASS}   // From environment variable — never hardcode!

  flyway:
    baseline-on-migrate: true   // Safe for existing DBs — baselines before migrating

app:
  jwt:
    secret: ${JWT_SECRET}   // Base64-encoded 256+ bit key
    expiration: ${JWT_EXPIRE}   // e.g. 86400000 for 24 hours
```

9.2 SQL Migration File

Place this file at: src/main/resources/db/migration/V1__create_users_table.sql

```
-- V1__create_users_table.sql
-- Flyway naming convention: V{version}__{description}.sql

CREATE TABLE IF NOT EXISTS users (
    id       BIGINT       AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50)  NOT NULL UNIQUE,
    email    VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,   // BCrypt hash is 60 chars — 255 is safe
    role     VARCHAR(20)  NOT NULL DEFAULT 'ROLE_USER',   // Matches Role enum values
    enabled  BOOLEAN      NOT NULL DEFAULT TRUE   // For account locking feature
);
```

9.3 Flyway Naming Rules

Part	Explanation
V	Prefix — 'V' for versioned, 'U' for undo, 'R' for repeatable
1	Version number — must be unique and increase (1, 2, 3... or 1.1, 1.2...)
__	Double underscore separator — required by Flyway
create_users_table	Description — human-readable, spaces replaced with underscores
.sql	Extension — Flyway only runs .sql files by default

9.4 How Flyway Works on Startup

- Flyway connects to the database on application startup
- It reads the flyway_schema_history table (created automatically)
- Compares applied migrations with files in db/migration/
- Runs any new V*.sql files in version order
- Marks each migration as applied with a checksum
- baseline-on-migrate: true — if no history table exists, it baselines at V0 first

10. Lombok Annotations Reference

Lombok generates boilerplate Java code at compile time via annotation processing. Here is what each annotation used in this project does:

Annotation	What Lombok generates
@Getter	public getX() method for every field
@Setter	public setX(X x) method for every field
@NoArgsConstructor	public ClassName() {} — zero-argument constructor
@AllArgsConstructor	Constructor with all fields as parameters (in declaration order)
@RequiredArgsConstructor	Constructor for all final fields and @NonNull fields — used for DI
@SuppressWarnings("unused")	Not Lombok — suppresses IDE 'unused method' warnings (JwtServiceImpl)

10.1 @RequiredArgsConstructor and Dependency Injection

```
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;   // final → included in generated
constructor
    private final CustomUserDetailsService service;   // final → included in generated
constructor
    // Lombok generates:
    // public SecurityConfig(JwtAuthFilter jwtAuthFilter, CustomUserDetailsService
service) {
    //     this.jwtAuthFilter = jwtAuthFilter;
    //     this.service = service;
    // }
}
```

Best Practice: Using constructor injection via @RequiredArgsConstructor is preferred over @Autowired field injection. It makes dependencies explicit, supports immutability (final), and simplifies unit testing (pass mocks via constructor).

11. Role-Based Authorization

11.1 Role Enum

```
public enum Role {
    ROLE_ADMIN,   // Spring Security expects 'ROLE_' prefix for hasRole()
    ROLE_USER   // hasAuthority() uses exact string — both work here
}
```

11.2 hasRole() vs hasAuthority()

Method	Explanation
hasAuthority("ROLE_ADMIN")	Exact string match — used in this project. Works with any string.
hasRole("ADMIN")	Spring auto-prepends 'ROLE_' prefix. Equivalent to hasAuthority('ROLE_ADMIN').

11.3 Method-Level Security with @EnableMethodSecurity

Because @EnableMethodSecurity is on SecurityConfig, you can protect individual methods:

```
@RestController
@RequestMapping("/admin")
public class AdminController {

    @GetMapping("/dashboard")
    @PreAuthorize("hasAuthority('ROLE_ADMIN')")    // Checked before method executes
    public String dashboard() {
        return "Admin Dashboard";
    }

    @GetMapping("/report")
    @PreAuthorize("hasAnyAuthority('ROLE_ADMIN', 'ROLE_USER')")    // Multiple roles
allowed
    public String report() {
        return "Report";
    }
}
```

11.4 How Role Gets into JWT

- User registers / logs in → AuthServiceImpl calls jwtService.generateToken(userDetails)
- JwtServiceImpl reads authorities → getFirstAuthority() → extraClaims.put('role', ...)
- JWT contains: { sub: 'username', role: 'ROLE_ADMIN', iat: ..., exp: ... }
- JwtAuthFilter extracts username from 'sub', loads UserDetails (with authorities) from DB
- Spring Security uses authorities from UserDetails — not from JWT claims — for authorization

12. Environment Setup & Running the Application

12.1 Required Environment Variables

Variable	Example Value & Notes
DB_PASS	yourSecureDBPassword — MySQL root password
JWT_SECRET	base64-encoded 256-bit key. Generate: openssl rand -base64 64
JWT_EXPIRE	86400000 = 24 hours in milliseconds. 3600000 = 1 hour.

12.2 Generating a Secure JWT Secret

```
# Generate a strong Base64-encoded secret (Linux/Mac):
openssl rand -base64 64

# Example output (use this as JWT_SECRET):
# 4y7A2pZ8mN3...base64string...==

# Set as environment variable (Linux/Mac):
export JWT_SECRET='your_generated_secret'
export DB_PASS='your_db_password'
export JWT_EXPIRE=86400000
```

12.3 Testing the API

```
# 1. Register a new user
curl -X POST http://localhost:8080/api/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"john","email":"john@test.com","password":"secret123"}'

# Response:
# { "token": "eyJ...", "tokenType": "Bearer", "username": "john", "role": "ROLE_USER" }

# 2. Login
curl -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"john","password":"secret123"}'

# 3. Access protected endpoint
curl http://localhost:8080/user/profile \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...'
```

12.4 Common Error Responses

HTTP Status	Cause & Fix
401 Unauthorized	Missing or invalid JWT. Check Authorization header format: 'Bearer <token>'
403 Forbidden	Valid JWT but insufficient role. User accessing /admin/** without ROLE_ADMIN.
400 Bad Request	Validation failed — check @NotBlank, @Email, @Size constraints in DTOs.
500 Internal Server Error	Often misconfigured JWT secret (not valid Base64) or DB connection failure.

13. Security Checklist

Use this as a review checklist before deploying to production:

- JWT secret is at least 256 bits and stored in environment variable (not hardcoded)
- BCrypt strength factor is 12 or higher
- HTTPS/TLS is configured on the server — JWT in plain HTTP is insecure
- JWT expiration is set appropriately — shorter = more secure (recommend 15min–1hr for access tokens)
- Refresh token mechanism is implemented for long-lived sessions (not in this codebase — consider adding)
- User input is validated with Bean Validation (@NotBlank, @Email, @Size) before processing
- Passwords are never logged — ensure logging config excludes request bodies on auth endpoints
- SQL uses parameterized queries (? placeholders) — JdbcTemplate handles this automatically
- existsByUsername and existsByEmail checks prevent duplicate registrations
- The /api/auth/** endpoint is the only public endpoint — all others require authentication
- Role enum stored as String in DB — check Role.valueOf() won't throw if DB has unexpected value
- enabled flag is respected in CustomUserDetailsService (UserDetails.isEnabled())